

PETCARE INTELLIGENCE

Implementación SaaS multi-tenant y seguridad

Documentación técnica del frontend Next.js, autenticación, sesiones, PostgreSQL, RLS, RPC, privilegios, producción y pruebas.

Rama: featured-desing

Commit funcional: 4e3ebac

Base remota: migraciones alineadas

Validación: 19/19 pruebas multi-tenant

Contenido

Guía de arquitectura, implementación, controles de acceso y operación.

1. Resumen ejecutivo	5
Resultado	5
2. Arquitectura resultante	5
2.1 Stack documentado	5
2.2 Fuente de verdad del tenant	6
2.3 Límites de confianza	6
3. Registro e inicio de sesión	7
3.1 Formulario de registro	7
3.2 Validaciones de registro	7
3.3 Flujo transaccional compensado de registro	7
3.4 Reparación de cuentas históricas al iniciar sesión	8
3.5 Inicio y cierre de sesión	8
3.6 Cookies y renovación	8
4. Modelo multi-tenant	9
4.1 Tablas protegidas	9
4.2 Backfill de datos históricos	9
4.3 Restricciones para filas nuevas	9
4.4 Sincronización de secuencias	10
5. Triggers de integridad tenant	10
5.1 tg_set_clientes_duenos_clinica	10
5.2 tg_set_mascotas_clinica	10
5.3 tg_set_inventario_clinica	10
5.4 tg_set_mascota_child_clinica	10
5.5 tg_set_detalle_insumos_clinica	11
5.6 Superficie de ejecución	11
6. Matriz final de RLS	11

6.1 Políticas efectivas

6.2 Vista vw_citas_hoy

7. RPC seguras y operaciones atómicas12

7.1 registrar_mascota_con_dueno

7.2 ajustar_stock

7.3 registrar_tratamiento

7.4 enviar_solicitud_demo

7.5 rls_auto_enable

8. Mínimo privilegio en la Data API13

8.1 Rol anon

8.2 Rol authenticated

8.3 Privilegios predeterminados

9. Server Actions y código del frontend14

9.1 Cliente Supabase

9.2 Acciones de autenticación

9.3 Acciones de mascotas

9.4 Acciones de inventario

9.5 Acciones de citas

9.6 Acciones de expedientes

9.7 Acciones de vacunas

9.8 Detalle de mascota

9.9 Solicitud de demo

10. Endpoints, respuestas y semántica HTTP16

11. Errores, logs y exposición de datos17

12. Seguridad de producción17

12.1 Headers

12.2 Docker

12.3 Variables requeridas

13. Cambios de interfaz18

13.1 Login

18

13.2 Landing

18

14. Migraciones

18

14.1 Procedimiento de despliegue

19

15. Pruebas y evidencia

19

15.1 Suite pgTAP

19

15.2 Validaciones realizadas

19

16. Índices agregados

20

17. Inventario de archivos modificados

20

Frontend y producción

20

Supabase

21

18. Riesgos residuales y siguientes etapas

21

18.1 RBAC fino

21

18.2 Validación de constraints históricas

22

18.3 Configuración hosted

22

18.4 Protección antiabuso

22

18.5 Observabilidad

22

19. Checklist para futuros cambios

22

20. Conclusión

23

1. Resumen ejecutivo

Durante esta sesión se corrigió la aplicación para que opere como un SaaS multi-tenant en el que cada clínica constituye un tenant independiente. El aislamiento ya no depende de que el navegador envíe correctamente un `id_clinica`: la identidad se obtiene de la sesión de Supabase Auth y la base de datos deriva el tenant mediante el perfil de `public.usuarios`.

Los cambios principales fueron:

1. Corregir el registro y el inicio de sesión.
2. Convertir al creador de una cuenta en `Administrador` de una clínica nueva.
3. Eliminar del registro público la selección de rol y el ingreso de un ID de clínica.
4. Crear y reparar automáticamente el perfil interno y la clínica al iniciar sesión.
5. Aplicar aislamiento por tenant a todas las tablas clínicas mediante RLS.
6. Derivar y validar `id_clinica` mediante triggers y relaciones canónicas.
7. Encapsular operaciones sensibles o compuestas en RPC atómicas.
8. Reducir los privilegios de `anon` y `authenticated` al mínimo necesario.
9. Evitar que vistas, funciones y tablas nuevas se expongan accidentalmente.
10. Proteger las sesiones con cookies `HttpOnly`, rotación y renovación.
11. Limitar errores y logs sensibles en producción.
12. Agregar headers de seguridad y una imagen Docker de producción sin root.
13. Reparar la navegación del logotipo y el componente visual del perro en la landing.
14. Incorporar 19 pruebas pgTAP específicas para aislamiento multi-tenant.

Resultado

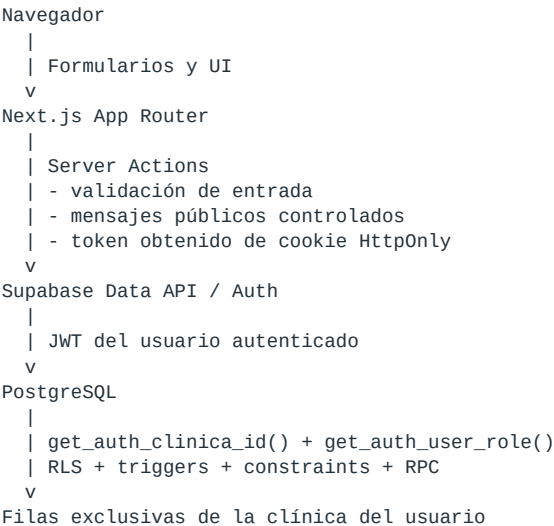
El aislamiento entre clínicas está implementado en profundidad: frontend, cliente autenticado, RLS, triggers, relaciones y RPC. Un usuario autenticado de la clínica A no puede consultar ni modificar filas de la clínica B, aunque manipule IDs en el navegador.

El RBAC fino todavía está deliberadamente incompleto. Las reglas especiales ya implementadas protegen cambios de clínica, eliminación de expedientes y cambios de estado de citas; la administración integral de usuarios por parte de propietarios se dejó para una etapa posterior.

2. Arquitectura resultante

2.1 Stack documentado

Componente	Versión o configuración
Next.js	<code>^16.2.12</code> , App Router
React	<code>19.2.4</code>
TypeScript	<code>^5</code>
Supabase JS	<code>^2.111.0</code>
PostgreSQL local	Major 17
Node.js	<code>>=22.0.0</code>
Contenedor	<code>node:22-alpine</code>



2.2 Fuente de verdad del tenant

La asociación canónica es:

```

auth.users.id
=
public.usuarios.id_usuario
->
public.usuarios.id_clinica
->
public.clinicas.id_clinica

```

Las políticas no confían en metadata modificable del navegador ni en un `id_clinica` recibido desde un formulario. Usan:

```

public.get_auth_clinica_id()
public.get_auth_user_role()

```

Estas funciones consultan `public.usuarios` usando `auth.uid()`.

2.3 Límites de confianza

Componente	Nivel de confianza	Responsabilidad
Navegador	No confiable	Capturar datos y presentar resultados
Server Actions	Confiable con validación	Validar formato, autenticar y orquestar
Cliente Supabase autenticado	Limitado por JWT	Ejecutar operaciones bajo RLS
<code>service_role</code>	Privilegiado	Registro, reparación y rollback de perfiles
PostgreSQL	Autoridad final	Aplicar RLS, integridad, locks y privilegios

La clave `SUPABASE_SERVICE_ROLE_KEY` se lee únicamente en módulos `server-only`. Nunca se envía al cliente ni se usa en componentes del navegador.

3. Registro e inicio de sesión

3.1 Formulario de registro

El formulario público solicita:

- Nombre completo.
- Correo electrónico.
- Contraseña.
- Confirmación de contraseña.
- Nombre de clínica.

Se eliminaron:

- Selector de rol.
- Campo `ID de clínica`.
- Flujo para que un usuario público se agregue como veterinario o recepcionista.

Todo creador de cuenta recibe:

```
rol = Administrador
id_clinica = clínica creada durante el registro
```

Esto evita que un visitante se autoasigne a una clínica existente o elija un rol arbitrario.

3.2 Validaciones de registro

`frontend/app/actions/auth.ts` aplica:

- Todos los campos marcados son obligatorios.
- Nombre personal y nombre de clínica: máximo 255 caracteres.
- Correo validado por formato.
- Contraseña: mínimo 8 caracteres, al menos una letra y un número.
- Coincidencia entre contraseña y confirmación.
- Detección de correo ya registrado.

`supabase/config.toml` alinea el entorno local con:

```
minimum_password_length = 8
password_requirements = "letters_digits"
```

3.3 Flujo transaccional compensado de registro

El registro ejecuta los siguientes pasos:

1. Crea el usuario en Supabase Auth con `signUp`.
2. Guarda `full_name` y `clinic_name` como metadata de recuperación.
3. Crea una fila en `public.clinicas` mediante el cliente administrativo.
4. Crea el perfil en `public.usuarios`.
5. Asigna siempre el rol `Administrador`.
6. Asigna el `id_clinica` recién creado.
7. Si existe sesión inmediata, escribe cookies y redirige a `/mascotas`.

- 8. Si la instancia exige confirmación de correo, devuelve éxito y espera el login posterior.

Si falla la clínica o el perfil, `rollbackRegistration()` elimina:

- Perfil parcial.
- Clínica parcial.
- Usuario de Supabase Auth.

PostgreSQL y Auth no comparten una sola transacción, por lo que este rollback compensado evita cuentas huérfanas en los fallos controlados.

3.4 Reparación de cuentas históricas al iniciar sesión

`ensureUserProfileHasClinic()` corrige tres estados:

1. Usuario Auth sin fila en `public.usuarios`.
2. Perfil existente sin `id_clinica`.
3. Creación concurrente del mismo perfil.

La función:

- Obtiene nombre y clínica desde metadata cuando están disponibles.
- Crea una clínica con nombre explícito o con el fallback `Clinica de <nombre>`.
- Crea o actualiza el perfil como `Administrador`.
- Elimina la clínica recién creada si el perfil no puede persistirse.
- Detecta si otra solicitud ya completó el perfil antes de reportar error.

3.5 Inicio y cierre de sesión

`loginAction()`:

1. Valida correo y contraseña.
2. Ejecuta `signInWithPassword`.
3. Repara perfil y tenant si hace falta.
4. Guarda la sesión en cookies seguras.
5. Redirige al panel de mascotas.

`logoutAction()`:

1. Recupera access y refresh token.
2. Revoca la sesión local en Supabase cuando es posible.
3. Elimina ambas cookies.
4. Redirige a `/login`.

3.6 Cookies y renovación

Cookies utilizadas:

Cookie	Uso	Protección
<code>sb-access-token</code>	JWT de acceso	<code>HttpOnly, SameSite=Lax, Secure</code> en producción
<code>sb-refresh-token</code>	Renovación	<code>HttpOnly, SameSite=Lax, Secure</code> en producción

`frontend/proxy.ts` inspecciona la expiración del JWT. Si faltan menos de cinco minutos, renueva la sesión con el refresh token y reemplaza ambas cookies. Si la renovación falla, elimina la sesión inválida.

El cliente del navegador tiene desactivados:

- `persistSession`.
- `autoRefreshToken`.
- `detectSessionInUrl`.

La sesión queda bajo control del servidor y no en `localStorage`.

4. Modelo multi-tenant

4.1 Tablas protegidas

Tabla	Columna tenant	Relación canónica
<code>clinicas</code>	<code>id_clinica</code>	Tenant raíz
<code>usuarios</code>	<code>id_clinica</code>	Perfil del usuario
<code>clientes_duenos</code>	<code>id_clinica</code>	Clínica autenticada
<code>mascotas</code>	<code>id_clinica</code>	Clínica y propietario
<code>inventario</code>	<code>id_clinica</code>	Clínica autenticada
<code>expedientes</code>	<code>id_clinica</code>	Clínica de la mascota
<code>citas</code>	<code>id_clinica</code>	Clínica de la mascota
<code>vacunas</code>	<code>id_clinica</code>	Clínica de la mascota
<code>detalle_insumos_expediente</code>	<code>id_clinica</code>	Clínica del expediente y producto
<code>solicitudes_demo</code>	No clínica	Sin acceso directo; solo RPC pública

4.2 Backfill de datos históricos

La migración de integridad completa tenants faltantes usando relaciones existentes:

- Propietario desde sus mascotas cuando todas pertenecen a una sola clínica.
- Expediente desde su mascota.
- Cita desde su mascota.
- Vacuna desde su mascota.
- Detalle de insumo desde su expediente.

Los perfiles históricos sin clínica reciben una clínica nueva y rol `Administrador`.

4.3 Restricciones para filas nuevas

Se agregaron constraints `CHECK (id_clinica IS NOT NULL) NOT VALID` a las tablas clínicas. El modo `NOT VALID`:

- Bloquea nuevas filas sin tenant.
- Evita interrumpir la migración por datos históricos que todavía requieran limpieza.
- Permite validar los datos previos posteriormente con `VALIDATE CONSTRAINT`.

4.4 Sincronización de secuencias

Antes de reparar perfiles se sincronizaron las secuencias `SERIAL` con el máximo ID de:

- Clínicas.
- Propietarios.
- Mascotas.
- Inventario.
- Expedientes.
- Detalle de insumos.
- Citas.
- Vacunas.

Esto corrigió colisiones de llave primaria causadas por cargas históricas que insertaron IDs explícitos sin avanzar las secuencias.

5. Triggers de integridad tenant

Las RLS controlan qué filas puede alcanzar un rol. Los triggers agregan una segunda defensa que deriva y compara tenants antes de escribir.

5.1 `tg_set_clientes_duenos_clinica`

- Completa `id_clinica` desde la sesión.
- Rechaza un tenant diferente con `OWNER_CLINIC_MISMATCH`.

5.2 `tg_set_mascotas_clinica`

- Completa el tenant desde la sesión.
- Verifica que el propietario exista.
- Alinea propietario y mascota.
- Rechaza cruces con `PET_OWNER_CLINIC_MISMATCH` o `PET_CLINIC_MISMATCH`.

5.3 `tg_set_inventario_clinica`

- Deriva la clínica autenticada.
- Exige un tenant.
- Rechaza `INVENTORY_CLINIC_MISMATCH`.

5.4 `tg_set_mascota_child_clinica`

Se reutiliza para:

- Expedientes.
- Citas.
- Vacunas.

La función obtiene el tenant desde la mascota, compara el valor recibido y confirma que la mascota pertenece a la clínica autenticada.

5.5 tg_set_detalle_insumos_clinica

Verifica simultáneamente:

- Existencia del expediente.
- Existencia del producto.
- Mismo tenant para expediente y producto.
- Mismo tenant que el usuario autenticado.

Así se evita descontar inventario de otra clínica o asociarlo a un expediente ajeno.

5.6 Superficie de ejecución

Los roles `anon` y `authenticated` no tienen permiso para invocar directamente estas funciones de trigger. Los triggers continúan ejecutándose internamente al producirse una escritura autorizada.

6. Matriz final de RLS

Todas las políticas se aplican al rol `authenticated`. `anon` no tiene políticas clínicas.

Tabla	SELECT	INSERT	UPDATE	DELETE
<code>clinicas</code>	Misma clínica	Servidor	Solo Administrador, misma clínica	Servidor
<code>usuarios</code>	Perfil propio; Administrador puede leer su tenant	Servidor	Servidor	Servidor
<code>clientes_duenos</code>	Mismo tenant	Mismo tenant	Mismo tenant	Mismo tenant
<code> mascotas</code>	Mismo tenant	Mismo tenant	Mismo tenant	Mismo tenant
<code>inventario</code>	Mismo tenant	Mismo tenant	Mismo tenant	Mismo tenant
<code>expedientes</code>	Mismo tenant	Mismo tenant	Mismo tenant	Solo Administrador
<code>citas</code>	Mismo tenant	Mismo tenant	Administrador o Veterinario	Solo Administrador
<code>vacunas</code>	Mismo tenant	Mismo tenant	Mismo tenant	Mismo tenant
<code>detalle_insumos_expediente</code>	Mismo tenant	Mismo tenant	Mismo tenant	Mismo tenant
<code>solicitudes_demo</code>	Sin acceso directo	Sin acceso directo	Sin acceso directo	Sin acceso directo

6.1 Políticas efectivas

Política	Regla
<code>Clinicas_Select_Tenant_Policy</code>	<code>id_clinica = get_auth_clinica_id()</code>
<code>Clinicas_Update_Tenant_Policy</code>	Tenant propio y rol <code>Administrador</code>
<code>Usuarios_Select_Tenant_Policy</code>	Perfil propio o Administrador del tenant
<code>ClientesDuenos_Tenant_Policy</code>	<code>FOR ALL</code> dentro del tenant
<code>Mascotas_Tenant_Policy</code>	<code>FOR ALL</code> dentro del tenant
<code>Inventario_Tenant_Policy</code>	<code>FOR ALL</code> dentro del tenant
<code>Expedientes_Select_Tenant_Policy</code>	Lectura del tenant
<code>Expedientes_Insert_Tenant_Policy</code>	Inserción del tenant
<code>Expedientes_Update_Tenant_Policy</code>	Actualización del tenant

Política	Regla
<code>Expedientes_Delete_Admin_Tenant_Policy</code>	Eliminación por Administrador
<code>Citas_Select_Tenant_Policy</code>	Lectura del tenant
<code>Citas_Insert_Tenant_Policy</code>	Inserción del tenant
<code>Citas_Update_Vet_Admin_Tenant_Policy</code>	Actualización por Administrador o Veterinario
<code>Citas_Delete_Admin_Tenant_Policy</code>	Eliminación por Administrador
<code>Vacunas_Tenant_Policy</code>	FOR ALL dentro del tenant
<code>DetalleInsumos_Tenant_Policy</code>	FOR ALL dentro del tenant

6.2 Vista `vw_citas_hoy`

La vista usa:

```
WITH (security_invoker = true)
```

Además filtra explícitamente por `get_auth_clinica_id()`. No tiene permiso para `anon`. Esto impide que una vista con privilegios de su propietario se convierta en bypass de RLS.

7. RPC seguras y operaciones atómicas

7.1 `registrar_mascota_con_dueno`

Responsabilidad:

- Crear propietario y mascota en una sola transacción PostgreSQL.
- Derivar `id_clinica` desde el JWT.
- Validar campos obligatorios.
- Evitar propietario huérfano si falla la mascota.

Seguridad:

- `SECURITY INVOKER`.
- Ejecutable solo por `authenticated`.
- Sin parámetro de tenant.

7.2 `ajustar_stock`

Responsabilidad:

- Aplicar un delta positivo o negativo.
- Bloquear la fila con `FOR UPDATE`.
- Evitar actualizaciones perdidas por concurrencia.
- Rechazar stock negativo.

Seguridad:

- `SECURITY INVOKER`.
- Producto limitado al tenant autenticado.
- No revela si un ID pertenece a otra clínica: responde `PRODUCT_NOT_AVAILABLE`.

7.3 registrar_tratamiento

Responsabilidad:

- Validar expediente, producto y cantidad.
- Confirmar que expediente y producto pertenecen al mismo tenant.
- Bloquear inventario.
- Crear detalle y descontar stock en una sola transacción.

Seguridad:

- Exige `auth.uid()`.
- Rechaza expedientes y productos de otros tenants.
- Ejecutable solo por `authenticated`.

7.4 enviar_solicitud_demo

Es la única RPC disponible para `anon`.

Controles:

- Valida nombre y correo.
- Normaliza el email.
- Limita longitudes.
- Impide otra solicitud del mismo correo durante 10 minutos.
- Inserta en una tabla sin privilegios directos para `anon` o `authenticated`.

7.5 rls_auto_enable

Función interna preparada para ser usada por un event trigger de PostgreSQL. No puede ejecutarse directamente desde la Data API.

En el estado inspeccionado no existe un objeto `EVENT TRIGGER` enlazado a esta función. Por tanto:

- Las nuevas tablas no reciben RLS automáticamente por esta función.
- Los privilegios predeterminados revocados evitan que nazcan expuestas a `anon` o `authenticated`.
- Toda migración futura debe activar RLS de forma explícita y agregar sus políticas.

8. Mínimo privilegio en la Data API

La migración final revoca los grants automáticos heredados.

8.1 Rol anon

Tiene:

- `USAGE` del esquema `public`.
- `EXECUTE` sobre `enviar_solicitud_demo(...)`.

No tiene:

- Acceso directo a tablas clínicas.
- Acceso a `solicitudes_demo`.
- Acceso a secuencias.

- Acceso a la vista de citas.
- Ejecución de funciones internas.

8.2 Rol authenticated

Tiene únicamente:

- CRUD en tablas clínicas generales, siempre sujeto a RLS.
- `SELECT`, `UPDATE` en `clinicas`.
- `SELECT` en `usuarios`.
- `SELECT` en `vw_citas_hoy`.
- `USAGE` en las secuencias necesarias para inserts permitidos.
- `EXECUTE` en las RPC autorizadas y helpers usados por RLS.

No tiene:

- `TRUNCATE`.
- Escritura directa en `usuarios`.
- Inserción o eliminación directa de clínicas.
- Acceso directo a `solicitudes_demo`.

8.3 Privilegios predeterminados

Se revocaron para futuras tablas, secuencias y funciones:

```
ALTER DEFAULT PRIVILEGES FOR ROLE postgres IN SCHEMA public
REVOKE ALL ... FROM anon, authenticated;
```

Cada objeto futuro debe declarar sus grants de forma explícita.

`supabase/config.toml` mantiene `auto_expose_new_tables` sin activar, alineado con el comportamiento seguro por defecto.

9. Server Actions y código del frontend

9.1 Cliente Supabase

Archivo	Responsabilidad
<code>frontend/lib/supabase/server.ts</code>	Clientes anon, admin y autenticado; lectura de cookies
<code>frontend/lib/supabase/client.ts</code>	Singleton del navegador sin persistencia de sesión
<code>frontend/lib/supabase/types.ts</code>	Tipos generados del esquema, relaciones y RPC
<code>frontend/lib/supabase/errors.ts</code>	Normalización controlada de errores
<code>frontend/lib/server-log.ts</code>	Logs de diagnóstico solo fuera de producción

`createAuthenticatedClient()` agrega el JWT del usuario a `Authorization`. Por eso cada consulta ejecuta las RLS correspondientes.

9.2 Acciones de autenticación

Función	Comportamiento
<code>loginAction</code>	Autentica, repara perfil, escribe cookies y redirige
<code>registerAction</code>	Crea Auth, clínica y perfil Administrador
<code>logoutAction</code>	Revoca sesión, elimina cookies y redirige
<code>ensureUserProfileHasClinic</code>	Repara cuentas históricas
<code>rollbackRegistration</code>	Compensa registros parciales

9.3 Acciones de mascotas

Función	Protección
<code>getMascotas</code>	Cliente autenticado; RLS filtra mascotas y propietarios
<code>registrarMascotaAction</code>	Validación y RPC atómica sin tenant del navegador
<code>eliminarMascotaAction</code>	Delete sujeto a RLS
<code>actualizarRecetaAction</code>	Update sujeto a RLS

9.4 Acciones de inventario

Función	Protección
<code>getCurrentUserProfile</code>	Valida JWT y rol conocido
<code>getInventario</code>	Lectura filtrada por RLS
<code>agregarProductoAction</code>	Trigger deriva tenant
<code>ajustarStockAction</code>	RPC con lock y límite de delta
<code>eliminarProductoAction</code>	Delete sujeto a RLS

9.5 Acciones de citas

Función	Protección
<code>getCitas</code>	Lecturas de mascotas, propietarios y citas bajo RLS
<code>agendarCitaAction</code>	Valida fecha y pertenencia de mascota
<code>cambiarEstadoCitaAction</code>	UI/servidor exige Veterinario o Administrador; RLS repite la regla

Solo permite transiciones desde **Pendiente** a **Completada** o **Cancelada**.

9.6 Acciones de expedientes

Función	Protección
<code>getExpedientes</code>	Lectura filtrada por tenant
<code>getMascotasDeClinica</code>	Opciones visibles solo del tenant
<code>registrarExpedienteAction</code>	Valida mascota antes de insertar
<code>eliminarExpedienteAction</code>	Exige Administrador y RLS confirma

9.7 Acciones de vacunas

Función	Protección
<code>getVacunas</code>	Lectura filtrada por tenant
<code>registrarVacunaAction</code>	Valida mascota, fechas y RLS

9.8 Detalle de mascota

`getMascotaDetalle`, `getExpedientes(id_mascota)` y `getVacunas(id_mascota)` validan IDs positivos y usan un cliente autenticado. Un ID de otro tenant produce `null` o una lista vacía porque RLS oculta la fila.

9.9 Solicitud de demo

`submitDemoRequest` valida longitudes y formato antes de llamar `enviar_solicitud_demo`. Los errores internos se convierten en mensajes públicos que no revelan PostgreSQL, SQL, nombres de tablas ni detalles de infraestructura.

10. Endpoints, respuestas y semántica HTTP

La aplicación no expone Route Handlers REST personalizados para estas operaciones. Usa Server Actions de Next.js y la Data API de Supabase.

Consecuencias:

- Las acciones devuelven estados de dominio `{ success, error }`.
- Los formularios no dependen de códigos HTTP inventados por el cliente.
- `redirect()` controla la navegación posterior a autenticación.
- Supabase/PostgREST conserva su semántica HTTP nativa.
- Los errores internos no se entregan directamente al navegador.

Para futuros Route Handlers, la convención recomendada es:

Caso	Código
Solicitud válida creada	201 Created
Solicitud válida sin contenido	204 No Content
Entrada inválida	400 Bad Request
Sin sesión	401 Unauthorized
Sesión válida sin permiso	403 Forbidden
Recurso inexistente o invisible por tenant	404 Not Found
Conflicto de estado o duplicado	409 Conflict
Validación semántica	422 Unprocessable Content
Límite de frecuencia	429 Too Many Requests
Fallo no controlado	500 Internal Server Error

No debe distinguirse entre “ID inexistente” e “ID de otro tenant” en respuestas públicas, porque esa diferencia permitiría enumerar recursos ajenos.

11. Errores, logs y exposición de datos

`reportServerError(scope, error):`

- Solo ejecuta `console.error` fuera de producción.
- En producción retorna antes de escribir.
- Reduce el error a un mensaje breve.
- Usa scopes internos para facilitar diagnóstico local.

Las Server Actions:

- Devuelven mensajes amigables.
- No devuelven objetos de error de Supabase.
- No incluyen SQL, stack traces, claves ni detalles de políticas.
- Usan mensajes neutros para recursos inexistentes o ajenos.

Esta decisión satisface el requisito de no dejar logs de depuración en producción. Como evolución, conviene integrar observabilidad estructurada con redacción de PII, muestreo y control de acceso, sin volver a exponer errores crudos.

12. Seguridad de producción

12.1 Headers

`frontend/next.config.ts` agrega:

- `Content-Security-Policy`.
- `Referrer-Policy: strict-origin-when-cross-origin`.
- `X-Content-Type-Options: nosniff`.
- `X-Frame-Options: DENY`.
- `Permissions-Policy`.
- `Strict-Transport-Security` solo en producción.

También:

- Desactiva `X-Powered-By`.
- Limita Server Actions a `256kb`.
- Usa salida `standalone`.

12.2 Docker

La imagen:

- Usa Node 22 Alpine.
- Instala dependencias con `npm ci`.
- Compila en una etapa separada.
- Copia únicamente el resultado `standalone`.
- Desactiva telemetría de Next.
- Ejecuta como usuario `nextjs`, no como root.

12.3 Variables requeridas

```
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
```

Reglas:

- La anon key puede llegar al navegador; su seguridad depende de RLS.
- La service role key es exclusivamente del servidor.
- No debe incluirse en `NEXT_PUBLIC_*`.
- No debe registrarse, enviarse a componentes cliente ni almacenarse en Git.

13. Cambios de interfaz

13.1 Login

- El logotipo “PetCare Intelligence” ahora es un enlace a `/`.
- `/` renderiza la landing principal.
- Se eliminó el selector de rol.
- Se eliminó el ID de clínica.
- Nombre de clínica permanece obligatorio.
- Contraseña muestra el requisito real de 8 caracteres, letra y número.
- Estados pendientes deshabilitan inputs y botón.

13.2 Landing

- Se restauró el activo `frontend/public/images/veterinary-dog-cutout.png`.
- El hero usa `next/image` con dimensiones y `sizes` estables.
- El CTA reutiliza la imagen con tamaño controlado.
- El layout responsive evita desbordes y superposiciones.
- La marca y los accesos hacia `/login` permanecen visibles.

14. Migraciones

Orden	Archivo	Propósito
1	<code>20260705225538_initial_schema.sql</code>	Esquema inicial y primeras políticas
2	<code>20260714210000_citas_module.sql</code>	Módulo de citas
3	<code>20260730203222_remote_schema.sql</code>	Alineación con cambios existentes en remoto
4	<code>20260730210000_multitenant_rls.sql</code>	Primera cobertura RLS integral
5	<code>20260730223000_enforce_multitenant_integrity.sql</code>	Backfill, triggers, políticas finales, índices y RPC
6	<code>20260730233000_saas_security_hardening.sql</code>	Secuencias, perfiles históricos, operaciones atómicas y demo
7	<code>20260731001500_least_privilege_data_api.sql</code>	Grants mínimos y defaults seguros

Las siete versiones quedaron alineadas entre el repositorio local y el proyecto Supabase remoto.

14.1 Procedimiento de despliegue

```
supabase migration list --linked
supabase db push --linked --dry-run
supabase db push --linked --yes
supabase db lint --linked
```

Siempre se debe revisar el `dry-run` antes de aplicar nuevas migraciones.

15. Pruebas y evidencia

15.1 Suite pgTAP

Archivo:

```
supabase/tests/database/multitenant_rls.test.sql
```

Resultado:

```
19 pruebas
19 aprobadas
0 fallos
```

Cobertura principal:

1. Tenant A solo lee su inventario.
2. Tenant A no recibe productos del tenant B.
3. Trigger rechaza escritura con otro tenant.
4. `authenticated` no puede crear perfiles.
5. `authenticated` no puede elevar su rol.
6. Clínica no se elimina desde Data API.
7. Vista de citas respeta el tenant.
8. Ajuste de stock opera dentro del tenant.
9. Stock actualizado persiste.
10. Ajuste de stock no alcanza otro tenant.
11. RPC de mascota crea propietario y paciente atómicamente.
12. Mascota deriva tenant desde JWT.
13. Propietario recibe el mismo tenant.
14. Administrador solo consulta usuarios de su clínica.
15. Tenant B obtiene únicamente su inventario.
16. Tenant B no recibe productos del tenant A.
17. Tenant B no lee la mascota creada por tenant A.
18. `anon` no puede consultar la vista clínica.
19. `anon` puede ejecutar únicamente la RPC pública de demo.

15.2 Validaciones realizadas

Verificación	Resultado
<code>npm run lint</code>	Aprobado
<code>npm run build</code>	Aprobado

Verificación	Resultado
TypeScript durante build	Aprobado
Imagen Docker de producción	Aprobada
Ejecución Docker sin root	Aprobada
<code>npm audit --omit=dev</code>	0 vulnerabilidades de producción
<code>supabase test db</code>	19/19
<code>supabase db lint --local</code>	Sin errores
<code>supabase db lint --linked</code>	Sin errores
Historial local/remoto	Alineado
Escaneo de secretos del commit	Sin credenciales detectadas

16. Índices agregados

Para evitar que las políticas y joins degraden con datos reales:

- `idx_usuarios_id_usuario_clinica`.
- `idx_clientes_duenos_clinica`.
- `idx_mascotas_clinica`.
- `idx_mascotas_dueno_clinica`.
- `idx_inventario_clinica`.
- `idx_expedientes_mascota_clinica`.
- `idx_citas_mascota_clinica`.
- `idx_vacunas_mascota_clinica`.
- `idx_detalle_insumos_expediente_clinica`.
- `idx_solicitudes_demo_email_created_at`.

17. Inventario de archivos modificados

Frontend y producción

Archivo	Cambio documentado
<code>frontend/.nvmrc</code>	Fija Node 22
<code>frontend/Dockerfile</code>	Build multi-stage y usuario no-root
<code>frontend/app/actions/auth.ts</code>	Registro, login, reparación, cookies y rollback
<code>frontend/app/actions/citas.ts</code>	Validación, tenant y roles
<code>frontend/app/actions/demo.ts</code>	Validación y RPC pública
<code>frontend/app/actions/detalle-mascota.ts</code>	Lecturas autenticadas y filtradas
<code>frontend/app/actions/expedientes.ts</code>	CRUD seguro y eliminación administrativa
<code>frontend/app/actions/inventario.ts</code>	Perfil, inventario y stock atómico
<code>frontend/app/actions/mascotas.ts</code>	Registro atómico y CRUD bajo RLS
<code>frontend/app/actions/vacunas.ts</code>	Validación de mascota y fechas
<code>frontend/app/globals.css</code>	Landing y perro responsive

Archivo	Cambio documentado
frontend/app/inventario/InventarioClient.tsx	Retiro de logs cliente
frontend/app/landing/page.tsx	Imagen estable y navegación
frontend/app/layout.tsx	Metadata y recursos globales
frontend/app/login/page.tsx	Registro simplificado y logo enlazado
frontend/app/mascotas/MascotasClient.tsx	Retiro de logs cliente
frontend/lib/server-log.ts	Logging solo en desarrollo
frontend/lib/supabase/client.ts	Cliente sin sesión persistida
frontend/lib/supabase/errors.ts	Mensajes controlados
frontend/lib/supabase/server.ts	Separación de clientes y secretos
frontend/lib/supabase/types.ts	Tipos actualizados del esquema y RPC
frontend/next.config.ts	Headers, límites y standalone
frontend/package.json	Versiones y requisito de Node
frontend/package-lock.json	Resolución reproducible
frontend/proxy.ts	Renovación de sesión

Supabase

Archivo	Cambio documentado
supabase/config.toml	Passwords, límites y exposición segura
supabase/migrations/20260730203222_remote_schema.sql	Alineación histórica
supabase/migrations/20260730210000_multitenant_rls.sql	Base de RLS multi-tenant
supabase/migrations/20260730223000_enforce_multitenant_integrity.sql	Integridad y políticas
supabase/migrations/20260730233000_saas_security_hardening.sql	Endurecimiento funcional
supabase/migrations/20260731001500_least_privilege_data_api.sql	Mínimo privilegio
supabase/tests/database/multitenant_rls.test.sql	19 pruebas de aislamiento

18. Riesgos residuales y siguientes etapas

18.1 RBAC fino

El aislamiento multi-tenant está completo, pero varias tablas usan `FOR ALL` para cualquier usuario autenticado del tenant. Antes de habilitar creación de veterinarios y recepcionistas debe definirse una matriz RBAC formal para:

- Crear, invitar, suspender y eliminar usuarios.
- Restringir eliminación de mascotas, propietarios, vacunas e inventario.
- Definir qué puede editar una recepcionista.
- Separar permisos clínicos de permisos administrativos.
- Auditar cambios de rol.

La creación de usuarios debe realizarse desde una Server Action administrativa o una función Edge protegida, nunca mediante inserción directa del navegador en `public.usuarios`.

18.2 Validación de constraints históricas

Los constraints `NOT VALID` protegen filas nuevas. Después de auditar y corregir datos históricos:

```
ALTER TABLE ... VALIDATE CONSTRAINT ...;
```

Debe ejecutarse tabla por tabla durante una ventana controlada.

18.3 Configuración hosted

Las migraciones SQL se desplegaron en Supabase remoto. Las opciones de `supabase/config.toml` describen y gobiernan el entorno local; configuraciones de Auth hosted como confirmación de correo, SMTP, CAPTCHA o restricciones de red deben verificarse también en el Dashboard de Supabase.

18.4 Protección antiabuso

La RPC de demo limita por correo durante diez minutos. Para exposición pública de alto tráfico conviene agregar:

- CAPTCHA.
- Rate limit por IP en CDN o gateway.
- Monitoreo de abuso.
- Política de retención y privacidad de leads.

18.5 Observabilidad

No se imprimen logs de error en producción. Para operación empresarial se recomienda una plataforma de observabilidad con:

- Redacción de PII.
- Correlation IDs.
- Acceso restringido.
- Retención definida.
- Alertas sin incluir tokens ni payloads clínicos.

19. Checklist para futuros cambios

Antes de agregar una tabla clínica:

- [] Incluir `id_clinica`.
- [] Definir relación canónica para derivar tenant.
- [] Agregar trigger de consistencia cuando dependa de otra entidad.
- [] Activar RLS.
- [] Crear políticas para cada operación requerida.
- [] Agregar índice que incluya el tenant.
- [] Revocar acceso de `anon`.
- [] Otorgar a `authenticated` solo privilegios necesarios.
- [] Agregar prueba cruzada entre tenant A y tenant B.

- [] Regenerar `frontend/lib/supabase/types.ts`.
- [] Ejecutar lint local y remoto.
- [] Revisar `supabase db push --dry-run`.

Antes de agregar una Server Action:

- [] Validar tipos, longitudes y rangos.
- [] Usar `createAuthenticatedClient()`.
- [] No aceptar `id_clinica` como autoridad.
- [] No usar `service_role` para CRUD clínico normal.
- [] Devolver mensajes públicos neutros.
- [] No registrar tokens, PII o errores crudos.
- [] Revalidar únicamente rutas afectadas.
- [] Probar IDs de otro tenant.

20. Conclusión

PetCare Intelligence quedó preparado como una base SaaS multi-tenant sólida:

- El tenant se deriva de la identidad autenticada.
- RLS impide cruces entre clínicas.
- Triggers protegen la integridad relacional.
- RPC atómicas reducen estados parciales y carreras.
- Los privilegios de Data API siguen el principio de mínimo acceso.
- El registro crea propietarios administradores sin permitir autoasignación de roles.
- Las sesiones se administran mediante cookies seguras.
- La producción incorpora headers, límites, Docker no-root y ausencia de logs de depuración.
- Las pruebas automatizadas demuestran aislamiento entre dos tenants.

El siguiente trabajo natural es implementar el módulo administrativo de usuarios y completar el RBAC por rol sin modificar el fundamento multi-tenant ya establecido.